

RepLLM: Toward Automatically Reproducing Network Research Results

Yining Jiang¹, Yunxin Xu¹, Wenyun Xu¹, Yufan Zhu², Rui Liu¹, Tangtang He², Haiying Huang¹, Letian Zhu¹, Qingyu Song^{1*}, Qiang Su¹, Lizhao You¹, Lu Tang¹, Wanjian Feng³, Yuchao Zhang⁴, Linghe Kong⁵, Qiao Xiang^{1*}, Jiwu Shu^{1,6}

¹Fujian Engineering Research Center of High-Performance Intelligent Computing Systems, School of Informatics, Xiamen University ²Xiamen University ³Yealink ⁴Beijing University of Posts and Telecommunications

⁵Shanghai Jiaotong University ⁶Tsinghua University

Abstract

Result reproduction of computer networking research is challenging as the scarcity of open-source implementations and the complexity of heterogeneous system architectures. Even though Large Language Models have demonstrated potential in code generation, existing code generation frameworks often fail to address the long-context constraints and intricate logical dependencies, which are vital in reproducing network systems from academic papers. Thus, we introduce *RepLLM*, an end-to-end multi-agent framework designed to automate code reproduction from paper content. *RepLLM* features a collaborative architecture comprising four specialized agents—Content Parsing, Architecture Design, Code Generation, and Audit & Repair, which are coordinated through *Shared Memory* mechanism to ensure global context consistency. With the enhancement of Structured Chain-of-Thought LLM reasoning and a sandbox-isolated static-dynamic debugging methodology, our framework effectively resolves semantic discrepancies and runtime errors, thereby improving reliable reproductions. Extensive evaluations on representative papers in top conferences demonstrate that *RepLLM* outperforms state-of-the-art system-level LLM frameworks in generating compile-ready and logically correct systems. Our results show that, with the aid of *RepLLM*, we can reproduce 95% of the original benchmarks within approximately two hours while reducing token consumption by up to 10% compared with state-of-the-art baselines.

CCS Concepts

• **Networks** → **Network experimentation**; • **Computing methodologies** → **Intelligent agents**; • **Software and its engineering** → **Software development techniques**.

Keywords

Network research reproducibility, Large language models, LLM-based Code Generation

ACM Reference Format:

Yining Jiang, Yunxin Xu, Wenyun Xu, Yufan Zhu, Rui Liu, Tangtang He, Haiying Huang, Letian Zhu, Qingyu Song, Qiang Su, Lizhao You, Lu Tang, Wanjian Feng, Yuchao Zhang, Linghe Kong, Qiao Xiang, Jiwu Shu. 2026. *RepLLM: Toward Automatically Reproducing Network Research Results*.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26, Denver, CO, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3829170>

In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3789240.3829170>

1 Introduction

Reproducibility is considered as the cornerstone of computer networking research, offering substantial value to both the academic and industrial communities. Beyond ensuring the reliability and validity of research findings, reproducible results provide the industry with a rigorous foundation for technology selection and system optimization [16].

Result reproduction is challenging due to the limited availability of artifacts and the complexity of re-implementation. Current reproduction strategies generally fall into two categories: utilizing executable prototypes provided by authors [48] or third parties [55], or undertaking implementation on the textual description of the original paper. However, most artifacts in networking are unavailable. Analysis reveals that even within top conferences such as SIGCOMM and NSDI, only 38.05% of articles published over the past decade include open-source implementations. Thus, researchers are often forced to rely on re-implementation from scratch. However, re-implementation of a network system is resource-intensive and requires a strong command of domain-specific knowledge. This high barrier to entry stands in sharp contrast to adjacent fields like machine learning, where standardized frameworks often allow results to be replicated in a few days [47].

To reduce reproduction costs, recent research has increasingly turned to Large Language Models (LLMs) for automated code generation, based on the proficiency in completion, debugging, and synthesis [14, 31, 35, 51, 54]. Notable progress has been made in “paper-to-code” generation [36], particularly within the machine learning (ML) [25] and biomedical [29] fields. However, existing LLM-based methods remain largely domain-specific and are ill-suited for re-implementing code in network systems research. Network research spans diverse areas, including protocols, algorithms, and system architectures, and lacks a unified programming paradigm. For example, systems such as NetChain[19], GRoot[20], Teal[46] differ in their architectures. By contrast, existing ML code generation approaches typically benefit from standardized programming frameworks, such as PyTorch [32] and TensorFlow [1].

While recent approaches have attempted to bridge this gap, they remain limited to specific scenarios in which code generation tasks are defined by well-structured inputs and outputs. For example, some studies focus on translating structured specifications, such as RFCs or configurations, into code [9, 51], which require structured

*Qingyu Song and Qiao Xiang are corresponding authors.

inputs that are rarely available in broader systems research. Wang et al. [42] restrict their scope to satellite networks. In addition, Xiang et al. [45] reproduce the code for four papers spanning diverse areas. However, their approach relies on domain-specific prompts that require exhaustive human interactions.

The re-implementation of network research code with LLMs remains challenging. First, networking papers are typically lengthy and difficult for LLMs to process comprehensively. Providing excessive context at once can degrade the performance of large models [28, 37]. Second, generating complex network codebases with LLMs remains difficult. Existing studies [7, 18, 52] show that generating complex, multi-file codebases in a single inference pass is beyond the capabilities of current LLMs. Network research often involves a hierarchy of interdependent submodules, which requires multi-stage reasoning [40].

Recent approaches have applied agent-based frameworks and tool-augmented coding systems, such as Claude Code CLI [4], to repository-level code generation. However, significant limitations remain. First, existing approaches are often inefficient because they involve redundant iterative reasoning and repeated context propagation across execution steps, leading to prohibitive computational costs. Our experimental results show that generating a thousand-line codebase with the state-of-the-art (SOTA) Claude Code CLI [4] consumes more than ten million tokens. Second, the lack of verification and error-recovery mechanisms substantially hinders reproducibility. Poesia et al. [33] demonstrate that syntactic and type-related errors frequently arise when generating multi-file codebases. Similarly, Hong et al. [17] show that when models generate program logic and data structures simultaneously, redundant or inconsistent type definitions often emerge across files. These limitations highlight the need for systematic designs that support problem decomposition, state consistency, reasoning, and structured code generation.

To address these challenges, we propose *RepLLM*, an explicit multi-agent coordination system designed for reproducing network research code. We decompose the reproduction task into different stages, hierarchically deploying LLM-based agents to synthesize robust code artifacts. Our system features a coordination mechanism that enforces a deterministic workflow, facilitating precise information exchange via a shared memory. Further, we implement a hierarchical repair strategy within isolated sandboxes: we first utilize static analysis to eliminate syntactic errors, followed by runtime debugging and semantic alignment to iteratively refine the generated code. By integrating external bug-fixing tools with a LLM, *RepLLM* ensures executable code generation.

The key contributions of this paper are as follows:

- **A Multi-Agent Framework for Automated Code Generation:** We propose *RepLLM*, a framework that orchestrates collaborative LLM agents to transform academic papers into executable code. By defining specialized roles—including *Content Parsing Agent*, *Architecture Design Agent*, *Code Generation Agent*, and *Audit & Repair Agent*—we decompose complex generation tasks into manageable sub-processes, ensuring distinct responsibilities for extraction, design, implementation, and optimization.
- **Efficient Context Management:** To facilitate robust multi-agent collaboration, we design an explicit Shared Memory

architecture that decouples information storage from the LLM reasoning process. We introduce a standardized data structure to manage both paper content and code hierarchies, coupled with a rule-based read-write protocol. This mechanism ensures data consistency and enables seamless context sharing across different stages of the generation pipeline.

- **Sandbox Isolated Hierarchical Refinement Methodology:** We introduce an iterative refinement mechanism that integrates static code analysis with dynamic execution feedback. This framework enables agents to detect syntax errors via static checks and resolve logical runtime failures through execution traces, significantly improving the pass rate and reliability of the generated code.
- We conduct extensive experiments to evaluate *RepLLM*. The results demonstrate that our approach significantly outperforms existing baselines in generating compile-ready and logically correct code from scientific literature. Moreover, with minimal human intervention, *RepLLM* achieves high-fidelity reproduction of the performance observed in official open-source implementations.

2 Motivation

This section identifies the challenge of reproduction of network research results. While the demand for reproducibility is high and LLMs are capable of code generation, current frameworks are too narrow in scope, limiting their applicability to network research.

2.1 Reproduction of Network Research is Imperative

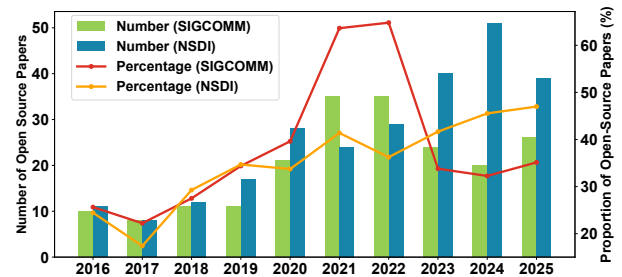


Figure 1: Statistics of SIGCOMM and NSDI papers with an open-source prototype from the authors (2016-2025).

Conducting a study of network systems necessitates a rigorous comparison with SOTA solutions, requiring substantial effort from authors to reproduce prior work. As highlighted in [45], 59.68% of papers compare their contributions against at least two existing systems, with authors manually reproducing an average of 2.29 systems per publication. Moreover, 49.20% of these studies involved reproducing at least one comparison system, while 26.65% required the reconstruction of two or more.

The manual reproduction of network system implementations presents several challenges. First, re-implementation imposes significant engineering overhead, diverting valuable time and resources away from novel research productivity. Second, the absence of source code or detailed experimental specifications can cause deviations from original results, undermining the credibility of comparative evaluations. Third, as modern network systems become

increasingly complex, manual verification becomes less tractable, slowing the validation of new work and detracting from innovation.

The limited availability of open-source implementations further exacerbates the overhead associated with reproducing experimental results. A statistical analysis of papers published in SIGCOMM and NSDI from 2016 to 2025, depicted in Figure 1, corroborates this observation: only 38.95% of SIGCOMM and 37.37% of NSDI papers made their source code publicly available during this period.

2.2 Challenges of LLM-based Code Reproduction for Network Research

The heterogeneity of the computer networking field presents three primary challenges to automated research reproduction:

- **Diverse Research Topics:** The field encompasses a wide array of distinct sub-domains (e.g., congestion control, traffic engineering, verification), challenging the domain-adaptability of LLMs.
- **Unstructured Research Formats:** Research papers lack a standardized format, diverse paper organizations, and multi-modal content like diagrams, challenging the interpretability of LLMs.
- **Heterogeneous System Architecture:** A network research work significantly varies from other proposed systems in system architecture or programming languages, thereby challenging the programming ability of LLMs.

The deficiency of existing methods for reproduction of network research motivates the development of our proposed new code generation frameworks.

2.3 Limitations of Existing LLM-based Code Generation Frameworks

Existing LLM-based code generation frameworks are designed either for general multi-task scenarios [4, 17, 22, 44] or domain-specific code generation frameworks [24, 29, 36, 56]. We analyze the existing limitations in detail, which motivate our fine-grained LLM-based framework to automatic code generation of system proposed in network research.

2.3.1 Limited Generalizability. Existing code generation frameworks are limited to specific domains. For example, Li et al. [24], Seo et al. [36] are for neural network (NN) models. [29] is designed for biomedical scenarios. These approaches use hard-coded domain-specific prompts for specific code generation. For example, in the NN model generation framework [24], PyTorch interfaces, e.g., `nn.ReLU`, are strictly defined in prompts.

2.3.2 Excessive LLM Token Consumption. Existing general-purpose multi-task frameworks employ varying numbers of LLM agents to address diverse tasks. Due to the reliance on implicit coordination among agents, these frameworks suffer from excessive token consumption and ambiguous task allocation. This structural limitation manifests in two key dimensions:

- **Inherent Redundancy in Multi-Agent Collaboration:** Redundant loading and processing within the context window of every agent leads to significant resource inefficiency. For example, AutoGen [44] utilizes broadcast mechanisms where identical messages are distributed to all participants.

- **Context Inflation in Long Tasks:** Existing multi-agent systems, like Claude Code CLI [4], have exhaustive token consumption with naive context appending. Moreover, this accumulation dilutes relevant information and degrades model performance [3, 44].

2.3.3 Inefficient Content Extraction and Management. For content extraction, LLMs still struggle to efficiently interpret complex and structured academic documents. Conventional parsers often lose semantics when extracting formulas, algorithms, and other non-textual elements [8]. Processing entire papers with long-context windows is also token-inefficient and prone to the “Lost in the Middle” effect, where models struggle to retrieve information embedded in dense contexts [27]. Moreover, accurately segmenting documents while preserving links between figures, equations, algorithms, and their textual descriptions remains a key bottleneck [13].

For context management, existing code generation approaches often lack explicit structural schemas, causing LLMs susceptible to coherence degradation and context attrition over extended interactions. Previous studies have emphasized the role of context management in keeping code generation organized and consistent [18]. Without structural constraints, LLMs may produce spurious file references, namespace collisions, circular dependencies, and redundant definitions. These problems collectively exacerbate hallucinations in LLM-based code generation [17, 34, 53].

Moreover, LLMs remain susceptible to inconsistent or fragmented code generation because existing methods primarily depend on prompt-driven alignment [17, 26, 34]. Prior studies show that expanding accessible context alone is insufficient to maintain long-range coherence [11, 27]. However, existing approaches often reduce context management to context-window expansion or retrieval optimization, without the structural consistency mechanisms required by systematic code generation.

To address these limitations in context management, we propose content abstraction and external memory mechanism to keep the extracted data aligned with the generated code.

2.3.4 Deficiency of Verification Mechanism. Generating code for large-scale network systems with LLMs in a “one-pass” manner is unreliable. Prior work [38] has shown that an iterative *Generate* → *Check* → *Repair* loop is more effective for improving reliability. In automatic paper reproduction, generated code commonly exhibits two types of issues: **syntax-level issues**, where errors prevent the code from compiling or executing correctly; and **semantic-level issues**, where the implemented algorithm deviates from the original paper, data-processing workflows are incorrect, or input/output formats are mismatched. Such semantic issues may allow the code to execute successfully but still fail to produce the expected results.

Existing code generation frameworks often lack robust post-generation verification and debugging mechanisms. As shown in [15, 49], current models fail to detect incomplete or incorrect solutions, resulting in generation without effective iterative refinement. Although recent approaches have improved syntactic correctness, they often miss deeper semantic errors. Furthermore, LLM-generated code rarely contains syntax errors but still exhibits subtle logic flaws and functional deviations that conventional syntax checkers cannot detect. [30, 43]

Moreover, existing verification mechanisms mainly target isolated, single-file unit tests and do not adequately capture repository-level dependencies. Although methods such as Self-Debug [10] leverage execution feedback, they cannot verify global consistency across interdependent modules. Without enforcement of architectural constraints, such as cross-module imports and symbol definitions, LLMs may generate code that is syntactically valid but structurally unbuildable [18].

These limitations motivate our runtime improvement framework. By combining static analysis with dynamic exception handling, our approach detects structural and runtime errors, refines implementation details, and improves the robustness of generated code.

3 System Overview

We introduce *RepLLM*, a multi-agent framework designed for automated code generation of network system research. *RepLLM* decomposes the workflow into a pipeline: 1) **Content Parsing**, 2) **Architecture Design**, 3) **Code Generation**, and 4) **Code Audit & Repair**. We design four collaborated agents to achieve efficient code generation. The overview of the pipeline code generation is shown in Figure 2. The pipeline proceeds as follows:

- **Content Parsing Agent (CPA)** transforms raw research documents into structured information, explicitly managing multimodal information rather than narrative text.
- **Architecture Design Agent (ADA)** decomposes the target system into a directed acyclic graph (DAG) of operational steps, establishing explicit I/O interfaces between consecutive steps.
- **Code Generation Agent (CGA)** synthesizes executable code for each step, using structured intermediate representations to bridge the semantic gap between natural language descriptions and program logic.
- **Audit & Repair Agent (ARA)** validates generated code through static analysis and runtime execution, feeding error feedback back into the generation loop for iterative refinement.

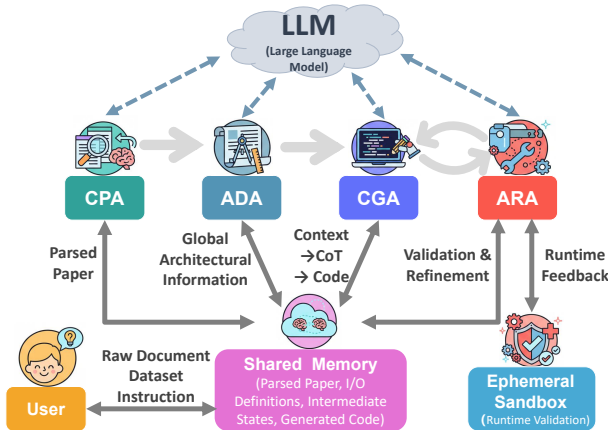


Figure 2: Overview of *RepLLM*'s Multi-Agent Pipeline.

Moreover, we design a **Shared Memory** to manage extracted data from paper and the generated content of the agents. It coordinates the pipelined stages of code generation, thereby preventing semantic drift across the whole workflow.

3.1 Workflow of *RepLLM*

The workflow of *RepLLM* is shown in Figure 3, where + denotes addition and ~ denotes modification.

3.1.1 Phase 1: Content Parsing & Initialization. The workflow begins with the User providing the raw research materials, including the manuscript (paper.md), dataset documentation (dataset.md), raw datasets (datasets/), and the user's instructions (Instruction). The CPA ingests these documents. It extracts multimodal content (figures, tables, algorithms) and preserves mathematical equations in standard $\text{\LaTeX}$ format. Then the CPA serializes the content into a structured JSON representation (paper.json and appendix.json). This creates a standardized, machine-readable index of the paper's sections, cross-references, and multimedia elements. These structured files are stored in the *Shared Memory*, serving as the ground truth for all subsequent agents.

3.1.2 Phase 2: Architectural Design. Once the content is parsed, ADA accesses the paper.json and dataset.md from *Shared Memory* to design the software architecture. ADA decomposes the reproduction task into a Directed Acyclic Graph (DAG) of discrete steps. It generates a detailed description for each step, including input/output data structures and dependencies. ADA outputs an arch.json file to *Shared Memory*. This file contains the workflow steps, dependencies (step_x), and mappings for data schemas (data_schema_mapping), and system information such as environment dependencies.

3.1.3 Phase 3: Code Generation Loop. Before writing functional code, CGA defines and generates the necessary class definitions and input/output interfaces, storing them in data_schema/.

The system enters a loop to generate code for each step defined in arch.json. For a specific step_x, CGA retrieves the relevant section context (section_sn) and step definitions from arch.json and paper.json. CGA first generates a Structured Chain-of-Thought (SCoT) (step_x.SCOT). This intermediate representation maps out the logical flow before actual coding begins. Guided by the SCoT and data structures, CGA writes the executable code for the step, saving it to the code/ directory.

The system immediately validates the generated code. If the code does not align semantically with the step description (detected via ARA), a repair loop is triggered. The agent reads the current code and context, fixes the discrepancies, and updates the files in code/ and data_schema/.

3.1.4 Phase 4: Audit, Repair, & Execution. After all steps are generated, ARA takes over to integrate, verify, and execute the code. ARA checks if the dataset loading logic is correct. It synthesizes a unified entry_point_file (e.g., main.py or bash script) to orchestrate the modular components. After that, ARA performs a paper-level semantic check. It verifies that the integrated system logic matches the overall methodology described in the manuscript. If misaligned, it modifies the code and entry point file. Before execution, ARA

runs static analysis tools (e.g., Ruff for Python) to detect and resolve syntax. Then, based on `arch.json`, ARA generates the Docker configuration, executes the code in a Docker container, and iteratively repairs errors identified during execution.



Figure 3: Workflow of RepLLM

4 Design of Agents

This section presents the detailed design of the agents and the *Shared Memory*.

4.1 Content Parsing Agent

CPA leverages LLM to reorganize raw document content into a structured JSON representation. This approach standardizes interactions with the LLM, thereby enhancing the efficiency and accuracy of multimodal content comprehension. All extracted data is persisted within *Shared Memory*.

4.1.1 Multimodal Content Extraction. We utilize the MinerU framework [41] to extract multimodal content, including figures, tables, algorithms, and equations, from the PDF file. Specifically, the framework preserves tables using standard Markdown format and references figures via generated image URLs. To ensure mathematical

fidelity, both equations and algorithms are retained in their native $\text{\LaTeX}$ representations. We transcode source document into Markdown formats, which facilitates JSON. By leveraging this tool, we are able to preserve information in a more comprehensive and faithful manner.

4.1.2 Content Organization. We index content globally for fast retrieval. We segregate auxiliary elements, i.e., figures, tables, algorithms, and equations, into distinct arrays, strictly preserving their original sequential order. Textual content is organized hierarchically: root-level entries correspond to top-level sections, with subsections nested within their respective parents. Bibliographic citations are explicitly excluded.

Moreover, for multiple cross-reference results in a research paper, e.g., figures and algorithms among sections, CPA explicitly maintains lists of referenced figures, tables, algorithms, and objects for extracted sections. When a downstream agent queries content for a specific section, CPA recursively expands all cross-referenced content, ensuring that the agent receives a self-contained context without implicit external dependencies. This design avoids a common failure: code generation references an assumption mentioned three sections earlier but not included in the context window

4.2 Architecture Design Agent

ADA orchestrates the structural decomposition and hierarchical organization of the generated codebase. ADA maintains architectural consistency by explicitly recording dependencies and file mappings in a unified schema, which guides all subsequent code generation phases. Further, we incorporate an external *Shared Memory* mechanism to manage LLM-generated content. It facilitates global code maintenance and ensures precise coordination across multiple files. Moreover, it significantly accelerates the generation process by reducing unnecessary token cost on redundant content.

4.2.1 Task Decomposition. First, ADA decomposes the comprehensive system reproduction task into discrete steps and utilizes distinct files to store step-wise code. We model the logical dependencies among steps as a Directed Acyclic Graph (DAG).

Within each step, ADA generates step description details, including data structure specifications of input/output (I/O), and a section identifier of related extracted context. For each step, inputs and outputs explicitly indicate their data sources, whether from the original dataset or outputs of preceding steps. These artifacts are serialized in JSON format and persisted in *Shared Memory*. ADA also leverages LLM to generate a high-level context summary, capturing the overall task, application domain, and expected I/O of target system from manuscript. Moreover, in order to constrain and facilitate downstream code generation, ADA incorporates a manual specification of the target programming languages.

4.2.2 Cross-File Dependency Management. ADA organizes content generated by LLMs through indices. We map LLM-generated artifacts, including source code files, I/O definitions, and their corresponding data schema definitions, with key-value pairs stored in *Shared Memory*. In subsequent code generation phases, ADA straightforwardly queries relevant context. This approach significantly reduces data retrieval overhead and preserves global coherence across the multi-file codebase.

4.3 Code Generation Agent

CGA generates code with two distinct phases: data schema definition and functional logic implementation. Before generating the core logic, *CGA* synthesizes the necessary data schema code based on the I/O specifications from *Shared Memory*. This preliminary step ensures interface consistency between steps and eliminates potential syntactic errors in advance. Further, *CGA* utilizes Structured Chain-of-Thought (SCoT) [23] to systematically implement the functional code for each step.

4.3.1 Code Generation Pipeline: Schema-first, logic-second. *CGA* first generates data schema definitions, such as TypedDict declarations for Python and struct/header definitions for C and P4, to precisely characterize the structure of data exchanged within each step. In this phase, field names, nested data structures, and type annotations are aligned with the dataset specification so that downstream data-loading components can parse actual inputs reliably and consistently.

Then, *CGA* implements each step by treating the schema as an explicit contract and standardizing output using the corresponding schema types. It is noteworthy that the strategy also addresses the I/O shifting problem in long-iteration LLM calling, where a data field's type or name diverges across steps.

4.3.2 Structured Code Translation (SCoT). We first employ SCoT to translate an algorithm into a structured notation, and subsequently translate this notation into target-language code. In this process, SCoT serves as an intermediate representation that bridges natural-language algorithm descriptions and executable code. Furthermore, for each code-generation step, we define operations over predefined data structures, where the input and output schemas explicitly specify the data dependencies required to coordinate the generation process.

SCoT captures the control flow of each step, such as sequential execution, conditional branching, and iterative looping, in a language-agnostic manner. By separating *what to do* in SCoT from *how to implement it* in code, SCoT reduces the reasoning burden on LLMs. In particular, the model no longer needs to concurrently interpret natural-language descriptions, determine program structure, and ensure data schema consistency during code generation.

4.4 Audit & Repair Agent

ARA achieves system orchestration and step-wise verification, leveraging a hybrid of static and dynamic repair strategies to ensure codebase consistency and correctness.

4.4.1 System Orchestration. *ARA* first generates a global entry script for the entire system. The LLM is prompted to generate this script based on the complete system architecture, the generated step-level file tree, and the dataset structure.

The generated script includes four primary components: First, it specifies a well-defined execution pipeline for all modules according to the DAG produced by *ADA*. Second, it includes commands for installing the necessary dependencies (e.g., pybatfish). Third, it provides command-line argument parsing to configure runtime parameters. Fourth, it performs pre-validation checks before execution. By combining these functions into a single entry script, *ARA* mitigates common failure modes caused by multiple entry points,

unclear inter-module dependencies, and mismatched runtime arguments.

4.4.2 Static Repair. Static repair has three steps, i.e., step-wise verification, system-level semantic alignment, and static syntactic repair.

Step-Wise Verification. *ARA* retrieves the generated code from *Shared Memory* and cross-validates it against the corresponding text for each step-wise code generation. If discrepancies are detected, *ARA* instructs LLM to update code with patches to enhance alignment with the specifications.

System-Level Semantic Alignment. *ARA* verifies the codebase's alignment with the corresponding text. It evaluates the semantic deviations between generated code (data schema definitions and step implementations) and the text description (paper content, appendix and dataset descriptions).

Static Syntactic Repair. To avoid unnecessary LLM invocations, *ARA* first applies off-the-shelf static analysis tools before resorting to LLM-based repair. To accommodate different programming languages, *ARA* provides language-specific analysis and repair interfaces. For example, for Python programs, we employ Ruff [6] to detect and automatically fix syntax issues.

4.4.3 Dynamic Repair. *ARA* utilizes an isolated sandbox to achieve dynamic repair, thereby enhancing robustness and safety. It has two steps, i.e., sandbox construction and iterative runtime repair.

Sandbox Construction. *ARA* leverages Docker [12] to construct isolated sandbox environments for reproducible execution. Firstly, it generates Docker configuration files according to the system architecture and the generated codebase. For systems that depend on third-party environments, such as p4-bmv2, *ARA* retrieves a pre-existing base image and configures the required runtime environment on top of it.

In addition, *ARA* embeds the generated entry script into the Docker image and assigns the necessary executable permissions. The workspace, dataset, and output directories are mounted into the container, allowing the sandboxed environment to execute the entry script under controlled conditions.

Iterative Runtime Repair. Iterative Runtime Repair is the final validation phase within *ARA*, where the generated system is executed end-to-end to catch errors that are invisible to static analysis. Unlike syntax errors, runtime errors emerge only during execution: missing dependencies, environment misconfigurations, I/O interface mismatches, and unhandled edge cases. The repair operates inside the Docker sandbox and iterates up to five times.

4.5 Shared Memory

We utilize *Shared memory* to manage information across all agents. It coordinates the pipelined code generation stages. We maintain a directory tree on disk. The shared memory holds three layers of information:

- **Paper layer:** structured paper content extracted during ingestion, including cross-referenced sections, figures, tables, equations, and supplementary materials.

- **Architecture layer:** the system architectural blueprint produced by the design stage, which defines steps, their interfaces, the precedence graph governing execution order and the environment dependencies.
- **Code layer:** the generated implementation produced by the code synthesis stage.

5 Implementation

This section presents the concrete mechanisms and algorithms that implement the design principles described in §4. We focus on the data structures, parsing algorithms, patch protocol, and runtime pipeline that constitute the reproducibility engine.

5.1 LLM API Interaction

We adopt a stateless LLM API interface in which LLMs do not receive the full dialogue history in subsequent requests. Thus, there is no conversation history across interactions. All information provided to LLMs via API calls is explicitly manipulated by *RepLLM*, thereby achieving zero token waste.

5.2 Shared Memory

First, we implement *Shared Memory* using the file system. Concretely, workspace contains a root-level `shared_memory` directory that is accessible to all agents. In addition to the two structured artifacts described below, the workspace also includes all generated source files, such as module implementations, data schemas, configuration files, and intermediate outputs. Later stages can read and write these files under the same access conventions.

The dedicated `shared_memory` directory is excluded from the file-tree construction during code generation. Thus, its intermediate representations are not exposed to LLMs as ordinary source files in prompts. This isolation improves both safety and robustness by preventing internal coordination artifacts from misinterpretation or modification as part of the generated codebase.

Second, we separate the `shared_memory` into two file spaces: 1) the paper space for the content extracted from the paper and 2) the system space for the reproduced system architecture and code.

For the paper space, we construct a `paper.json` file to store the extracted content in a structured tree. This tree organizes sections, figures, tables, algorithms, and equations, and is subsequently extracted into prompts for LLM-based generation. Each section is assigned a unique identifier, which enables efficient access to relevant paper content. Correlated content is recursively linked through cross-references. In addition, an optional `appendix.json` file is attached to the section specified as the source of each generation step.

For the system space, we construct an `arch.json` file generated by *ADA*. This file records the reproduced system architecture, including step decomposition, step-level workflows, interfaces, numeric contracts, and the environment dependencies. `arch.json` is updated in place during reproduction: data-schema generation appends schema mappings for each step, code generation writes per-step SCoT fields, and audit/repair generates the script for system entrance.

To handle incomplete intermediate states, missing files or referenced fields are replaced by all available results rather than treated

as fatal errors. This design allows subsequent pipeline stages to remain executable while preserving a consistent system-level state.

Moreover, we implement `shared_memory` as a coherent and agent-agnostic state layer. To avoid expired state, reads are in parallel with writes and are always resolved lazily from the latest on-disk content rather than from cached copies. When the resolved files need to be used, they are wrapped as fenced and language-tagged blocks and then embedded into LLM prompts. This design improves consistency across agents while reducing the risk of stale context being propagated through the generation pipeline.

5.3 SCoT

Algorithm 1 SCoT Extraction from Paper Sections

```

1: function EXTRACT-SCoT(step s, paper P, dataset description
   D)
2:    $c_{\text{section}} \leftarrow \text{Recursive-Context-Retrieval}(P, s.\text{section\_sn})$ 
3:    $\text{ctx} \leftarrow s.\text{description} \oplus c_{\text{section}} \oplus D$ 
   // Stage 1: generate SCoT from natural language
4:    $\text{SCoT}_{\text{raw}} \leftarrow \text{LLM}(\text{ctx}, \text{SCoT\_prompt\_templates})$ 
5:    $\text{SCoT}_{\text{json}} \leftarrow \text{Extract-JSON}(\text{SCoT}_{\text{raw}})$ 
   // Stage 2: generate code from SCoT
6:    $\text{schema} \leftarrow \text{Get-Data-Schema}(s.\text{step\_index})$ 
7:    $\text{code}_{\text{raw}} \leftarrow \text{LLM}(\text{SCoT}_{\text{json}} \oplus \text{schema})$ 
8:    $\text{code}_{\text{final}} \leftarrow \text{Parse-JSON-Block}(\text{code}_{\text{raw}})$ 
9:   return ( $\text{SCoT}_{\text{json}}, \text{code}_{\text{final}}$ )
10: end function
```

SCoT performs a two-stage translation: first from section text to structured SCoT, then from SCoT to executable code. Applied in *CGA*, Algorithm 1 illustrates the two-stage process. Given a step information, the paper, and the dataset description, it first assembles the context by retrieving the relevant section content. Stage 1 prompts the LLM to generate a raw SCoT, then parses it into a JSON structure and writes it into *arch.json*. Stage 2 feeds the SCoT together with the required data schema back to the LLM to produce raw code, which is finally parsed into executable code. The algorithm returns the SCoT along with the final generated code.

SCoT explicitly provides formal interface contracts (i.e., typed input/output specifications) and orchestrates the underlying logic using language-agnostic control flow notations, such as sequential steps, explicit branching, and loop blocks.

Moreover, SCoT abstracts algorithmic behavior through highly structured natural language, rather than mimicking programming syntax with mathematical notation, language-specific API conventions, or implicit data structures as in traditional pseudo-code. From an architectural perspective, SCoT provides an intermediate representation that bridges natural-language algorithm descriptions and concrete source code.

Figure 4 illustrates an example of SCoT for Mininet topology parsing. It first specifies the typed input and output of the step, where `topo_file` denotes the path to the topology description and `topo_obj` denotes the constructed Topo object. Then, it describes the logic as an ordered sequence of structured operations, including importing the required topology abstraction, defining a custom topology class, reading the topology file, parsing each line into

node pairs, and adding the corresponding links. The loop and conditional branches explicitly capture the control flow, while avoiding commitment to low-level implementation syntax.

Moreover, line 4 uses the language-agnostic expression “Open `topo_file` for reading,” which does not constrain the second-stage code generator to a specific implementation, such as Python’s `with open` or C++’s `ifstream`. In this way, SCoT keeps the fundamental algorithm and data dependencies while leaving language-specific realization to the code generation stage.

```
Input: topo_file: string representing the path to topo.txt
Output: topo_obj: an instance of Mininet Topo class
1: Import Topo from mininet.topo
2: Define a class CustomTopo that inherits from Topo
3: Define the build method in CustomTopo that takes
   topo_file as an argument
4: Open topo_file for reading
5: for each line in the file do
6:   Strip whitespace and split the line into a list of
   tokens
7:   if tokens is empty then
8:     Continue to the next line
9:   else
10:    Call addLink(tokens[0], tokens[1])
11: Create an instance of CustomTopo with topo_file and
   assign it to topo_obj
12: return topo_obj
```

Figure 4: An Example of SCoT for Mininet Topology Parsing

5.4 Structured Data Extraction from LLM Responses

RepLLM reliably extracts machine-readable structured data and code from free-form text of LLM responses. We define a lightweight XML-like tagging protocol using three tag types: `<json>` for structured data, `<file>` for full source file content, and `<patch>` for targeted text replacements. All tags are case-insensitive, accept optional attributes in flexible order, and support HTML-escaped variants (`<`; `json>`).

To robustly parse LLM outputs, *RepLLM* avoids brittle regex matching and instead uses a character-level scanner that recognizes closing tags only outside quoted strings. JSON extraction is performed through progressive fallbacks, including tagged blocks, HTML-unescaped tags, fenced code blocks, and bare JSON streams, with automatic repair for common LLM formatting errors. File and patch blocks are normalized before use, and patches are applied only when the target text is uniquely matched. Moreover, multiple edits to the same file are committed transactionally, with SHA-256 deduplication preventing duplicate applications.

6 Evaluation

In this section, we compare our proposed framework with SOTA code generation approaches. To demonstrate the practical utility and robustness of our method, we present experimental results centered on the reproduction of network research papers published in top-tier academic conferences.

Table 1: Selected Papers

System	Conference	Area	Lang.	Tools
NCFLOW [2]	NSDI'21	Traffic Eng.	Python	Gurobi
GRooT [20]	SIGCOMM'20	DNS Verif.	C++	–
Rateless IBLT [50]	SIGCOMM'24	Coding Theory	Go	–
SelfStarter [21]	NSDI'20	Config Mining	Python	PyBatfish
NetDice [39]	SIGCOMM'20	Prob. Verif.	Python	–
Teal [46]	SIGCOMM'23	Traffic Eng.	Python	CUDA
NetChain [19]	NSDI'18	In-network Sys.	Python&P4	Mininet

6.1 Baselines

To demonstrate the effectiveness of the *RepLLM* framework, we conduct a comparative evaluation against two representative baselines with distinct characteristics:

Standalone LLM: We directly feed the entire raw documents to a LLM and prompt it to generate the complete codebase in a single pass.

Claude Code CLI [4]: Claude Code CLI is a command-line tool that provides AI-powered coding assistance, supporting autonomous file operations and bash execution. We employ Claude Code CLI to achieve multi-agent-based code generation.

We deploy Claude Code CLI in a command-line environment and configure multiple agents for code generation, debugging, and refactoring, respectively. To ensure full automation, we strictly constrain human involvement during the generation process: human operators only approve execution and file-modification requests, without providing additional guidance, hints, or manual intervention. We evaluate Claude Code CLI under three configuration modes, namely low, medium, and high, to assess its code generation performance.

6.2 Setup

For the LLM API, we use Gemini-3.1-Pro-preview as the backbone model. We employ Docker to conduct sandboxes. All experiments are conducted in a controlled environment equipped with a standard development stack and the necessary runtime dependencies. To improve evaluation stability and reduce run-to-run variance, We repeat each experiment on *RepLLM* three times and report the aggregated results.

6.2.1 Selected Research Paper. Our framework can generalize to any system-level paper with sufficient implementation detail, regardless of venue. Here we select seven representative papers spanning diverse networking research domains and architectures. As shown in Table 1, these works cover traffic engineering, DNS verification, coding theory, configuration mining, probabilistic verification, and in-network systems, and span implementations originally written in Python, C++, Go, and Python&P4. To eliminate language-induced variance, we set the implementation of all frameworks and *RepLLM* exclusively to Python and P4.

6.2.2 Evaluation Metrics. We define a multi-dimensional set of metrics to evaluate the code generation performance of all methods:

- **Code-Level Reliability:** This metric assesses the syntactic correctness and execution readiness of the generated code. It consists of two sub-metrics: 1) *Executability* and 2) *Code-Level Data Loading Success*, which verifies whether the generated code is able to load the open-source datasets referenced in the original paper.

Table 2: Evaluation of Code Executability and Dataset Loading

	Standalone LLM		Claude Code CLI		RepLLM (ours)	
	Runnable	Dataset Loaded	Runnable	Dataset Loaded	Runnable	Dataset Loaded
Rateless IBLT	✗	✗	✓	✓	✓	✓
GRooT	✓	✓	✓	✓	✓	✓
NCFlow	✓	✓	✓	✓	✓	✓
SelfStarter	✓	✓	✓	✓	✓	✓
NetDice	✓	✗	✓	✓	✓	✓
Teal	✗	✗	✓	✓	✓	✓
NetChain	✗	✗	✓	✓	✓	✓

- **System-Level Paper–Code Semantic Alignment:** This metric evaluates the structural and logical fidelity of the generated system with respect to the paper description. We adopt an LLM-based evaluation protocol to assess three aspects: completeness (Comp.), correctness (Corr.), and maintainability (Maint.).
- **Code Generation Cost:** This metric quantifies the computational and monetary cost incurred during code generation. We report the input tokens, output tokens, and cumulative token consumption across the entire generation process.

We further evaluate whether the generated code can reproduce the reported results. In this evaluation, we allow human intervention to correct semantic mismatches between the generated code and the target functionality.

- **Reproduction Performance:** We measure the consistency between the experimental results produced by the generated code and two reference sources: 1) the results reported in the original paper, and 2) the results obtained by executing the official open-source implementation.
- **Reproduction Calibration Cost:** We quantify the human effort required to refine the generated code into a valid reproduction. We measure the total refinement time (time-to-completion) and the number of modified lines (diff size), using Cursor IDE [5] for code editing.

6.3 Results of Code-Level Reliability

As shown in Table 2, *RepLLM* achieves both executable and successful dataset loading on all tasks. In contrast, the Standalone LLM baseline fails on several tasks, including Rateless IBLT, Teal, and NetChain, where it cannot execute the system or properly integrate datasets, highlighting its limitations in robust environment reconstruction.

Moreover, Claude Code CLI also executes all tasks without runtime errors under three modes; however, it does not faithfully reconstruct dataset loading configurations. It usually provides oversimplified implementations or implicit assumptions of data rather than accurate integration, thereby limiting the availability to real datasets. For example, in NetChain, Claude Code CLI produces a pure Python state-machine simulation, which entirely bypasses the BMv2 and Mininet.

6.4 Results of Paper–Code Semantic Alignment

Table 3 summarizes the average paper–code semantic alignment scores across the seven benchmark papers. Overall scores are judge-provided holistic ratings and are not computed from the preceding three dimensions. The final column reports the equal-weighted

average of the three judge-provided Overall scores. The three LLM judges exhibit different calibration scales: GPT-5.5 assigns more conservative absolute scores, whereas Gemini-3.1-Pro-Preview tends to give higher scores.

Moreover, we report the equal-weighted cross-judge overall score and further characterize inter-rater reliability. We use Kendall’s W to measure rank agreement and ICC(3,1) with ICC(3,k) to measure consistency for a single judge and the averaged multi-judge score, respectively.

As shown in Table 4, the three judges show consistent evaluation behavior despite their different score scales. For the holistic overall score, Kendall’s W reaches 0.747, indicating substantial rank agreement among the judges. The consistency-based ICC values further support this conclusion: ICC(3,1) is 0.593 for a single judge, while ICC(3,k) increases to 0.814 when aggregating the three judges. These results suggest that the relative quality assessment is stable across heterogeneous LLM evaluators, and the final averaged score is not dominated by the bias of a single judge.

RepLLM demonstrates strong reliability across repeated runs. Across three independent generations, *RepLLM* obtains a cross-judge averaged overall score of 6.18 ± 0.24 , showing low run-to-run variance. This stability is important because paper-to-code generation is inherently open-ended: a reliable system should not depend on a single favorable generation. The small standard deviation indicates that *RepLLM* repeatedly produces repositories with comparable semantic alignment quality.

RepLLM also consistently improves over standalone LLM generation. Its averaged Overall score is 6.18, compared with 5.40 for Standalone LLM, corresponding to a 0.78 point improvement. This gain, together with the low variance across repeated runs, indicates that *RepLLM* provides a more dependable generation process than directly prompting an LLM to generate the repository in one pass.

Overall, the results show that *RepLLM*’s advantage lies not only in its score level but also in its robustness. The multi-judge agreement analysis supports the reliability of the LLM-based evaluation, while the repeated-run results show that *RepLLM* can stably produce semantically aligned code across independent executions.

6.5 Results of Code Generation Token Cost

Figure 5 reports token consumption across baselines. For *RepLLM*, we report mean $\pm$ standard deviation to show run-to-run consistency. From the results, baseline methods either generate little code or require very large input budgets. In contrast, *RepLLM* achieves a better balance between input cost and effective code generation. The result demonstrates that *RepLLM* is more efficient in system-level code generation with a given budget.

Standalone LLM consumes few tokens, but they also produce limited code. For example, on GRooT, the standalone LLM uses only 38.4K input tokens and generates 10.3K output tokens. This low cost mainly reflects limited system coverage rather than high efficiency.

Claude Code CLI incurs substantially higher input costs across benchmarks. On NetDice, Claude Code CLI requires roughly 9M input tokens, respectively. These large input budgets reflect repeated context processing across different modes, yet the final output remains much smaller than that of *RepLLM*.

Table 3: Average LLM-as-a-judge scores across seven papers.

Method	GPT 5.5				Gemini-3.1-Pro-Preview				DeepSeek-V4-Pro				Cross-Judge
	Corr.	Comp.	Maint.	Overall	Corr.	Comp.	Maint.	Overall	Corr.	Comp.	Maint.	Overall	Avg. Overall
Standalone LLM	3.00	5.29	4.57	4.04	7.00	7.86	6.71	7.26	4.29	5.86	5.00	4.89	5.40
Claude Code CLI(low)	3.00	5.57	5.29	4.27	7.86	9.29	8.07	8.29	5.57	7.14	6.14	5.80	6.05
Claude Code CLI(medium)	3.00	5.43	5.43	4.46	7.43	9.00	8.29	7.90	6.57	8.29	7.29	6.67	6.34
Claude Code CLI(high)	3.14	5.57	5.29	4.47	7.86	9.00	8.86	8.29	5.57	7.57	6.57	6.00	6.25
Source Code	5.71	7.00	6.57	6.44	9.43	9.43	9.31	9.43	8.86	9.29	8.43	8.96	8.24
<i>RepLLM</i> (avg. $\pm$ std.)	3.10 $\pm$ 0.22	5.33 $\pm$ 0.30	4.81 $\pm$ 0.08	4.31 $\pm$ 0.19	8.10 $\pm$ 0.36	8.86 $\pm$ 0.14	7.52 $\pm$ 0.87	8.16 $\pm$ 0.47	5.52 $\pm$ 0.54	7.10 $\pm$ 0.30	5.81 $\pm$ 0.33	6.06 $\pm$ 0.52	6.18 $\pm$ 0.24

Table 4: Inter-rater reliability among GPT-5.5, Gemini-3.1-Pro-Preview, and DeepSeek-V4-Pro judges.

Metric	Corr.	Comp.	Maint.	Overall
Kendall's <i>W</i>	0.666	0.595	0.649	0.747
ICC(3,1)	0.559	0.528	0.517	0.593
ICC(3,k)	0.791	0.771	0.763	0.814

Table 5: Cost Analysis of Code Generation and Repair

	Generation Time (min)	Human Fixing Time (min)	Total Time (min)	Modified Lines
GRooT	50	76	126	1535
NCFLOW	42	93	135	53
Rateless IBLT	38	84	122	428
NetDice	63	66	129	764
SelfStarter	41	41	82	208
Teal	59	23	82	232
NetChain	84	54	138	1174

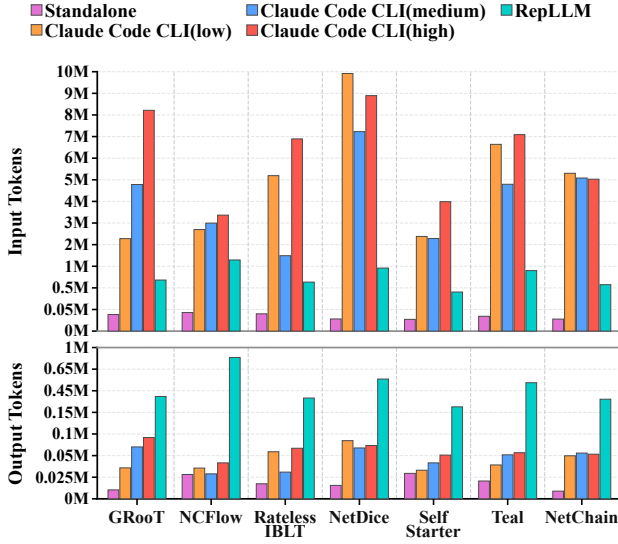


Figure 5: Code Generation Token Cost of Different Frameworks.

In contrast, *RepLLM* keeps input consumption substantially below Claude Code CLI while producing much larger implementation-oriented outputs. For instance, on NetDice, it generates 558.4K output tokens with 958.1K input tokens. These outputs include not only executable code, but also structured intermediate artifacts such as pseudo-code, architecture design, patches for improvement, and sandbox execution scripts.

6.6 Reproduction Calibration Cost

Table 5 reports the reproduction cost of *RepLLM*, measured by code generation time, human repair time, total time, and modified lines. The automated code generation stage takes 38–84 minutes. The remaining human repair takes 23–93 minutes. Across all benchmarks, end-to-end reproduction of results finishes within 82–138 minutes. Most of the later human modifications were needed because the paper describes the protocol and algorithms, but does not provide

full artifact-level implementation details. For example, in NetChain, performance-analysis scripts had to be added manually because the paper reports evaluation results but does not specify the exact scripts. Only a few changes were due to semantic mismatches, such as the exact NetChain wire-field order: the paper gives only a schematic packet-format diagram, not a byte-level implementation specification. The results show that *RepLLM* can quickly provide a strong implementation baseline and make full-system paper reproduction practical with modest human effort.

6.7 Analysis of Reproduced and Official Implemented Performance

We analyze the reproduction performance of our proposed *RepLLM* against official implementations. Due to the page limit, the detailed analysis and results for Claude Code CLI and standalone LLM are in Appendix §A.1.

6.7.1 Algorithm-only systems: GRooT, Rateless IBLT, and NetDice. Compared with standalone LLM baselines and Claude Code CLI, *RepLLM* better recovers paper-specific system structure without any function-level approximation. Although few semantic inconsistencies remain, after manual auditing and correction, *RepLLM* achieves substantially “perfect” reproductions: GRooT matches the benchmark violation profile on reference tests, Rateless IBLT validates the real protocol end to end, and NetDice passes all tests after correcting hot-edge reasoning, ECMP semantics, and bounded-exploration logic.

6.7.2 Third-party-tool-dependent systems: NCFLOW and SelfStarter. Compared with the baselines, *RepLLM* more accurately reconstructs the required system decomposition. The remaining errors mainly arise from external-tool interfaces and ambiguous data formats. This is caused by the gap between the paper-level description and the implementation-level data interface. After manual correction, NCFLOW achieves near-identical total flow on the main dataset,

and SelfStarter matches the expected ACL-level profile, including template counts, group outliers, parameter-level discrepancies, and final consistency summaries.

6.7.3 Hardware-dependent system: Teal. Compared with the standalone LLM and Claude Code CLI baselines, *RepLLM* better preserves the official system design, whereas the baselines either fail during data loading or reduce the learning task to simplified surrogate experiments. The remaining limitations mainly stem from framework and hardware constraints. After manual correction, *RepLLM* faithfully executes the FlowGNN-COMA-ADMM pipeline on B4, completes UsCarrier when path caches are available, produces valid split ratios, and reports the expected traffic-engineering metrics, including measurable ADMM gains over policy-only allocation.

6.7.4 Distributed runtime system: NetChain. NetChain requires a P4 data plane, a Python control plane, clients, and orchestration scripts to run jointly on Mininet/BMv2. This setting stresses distributed component closure: all modules must be present, share consistent topology and protocol state, and execute within the intended runtime environment.

RepLLM is the only approach that approaches true end-to-end NetChain reconstruction. Although *RepLLM* initially generates a four-module framework with imperfect client and protocol logic, the human-corrected version fixes lifecycle management and resource cleanup, and successfully runs packet exchange on the BMv2 testbed with consistent client/receiver throughput logs.

6.8 Ablation Studies

We conduct ablation studies to evaluate the effectiveness of our three core components: SCoT, Shared Memory, and ARA on reproducing SELFSTARTER.

Table 6: Ablation Study Results

Component	Runnable	Dataset Loaded
w/o SCoT	✓	✗
Pseudocode	✗	✗
w/o Shared Memory	✗	✗
w/o Architecture Memory	✓	✗
w/o Structured Paper Memory	✓	✓
w/o Interface Memory	✗	✗
w/o ARA (Static)	✓	✓
w/o ARA (Dynamic)	✗	✓

6.8.1 Impact of SCoT. We compare the full pipeline with two ablations: *w/o SCoT*, which removes SCoT, and *Pseudocode*, which replaces it with conventional pseudo-code interleaved with code. Both variants discard data schemas, interface contracts, and per-step algorithm traces. Consequently, neither reproduces SELFSTARTER end to end, despite consuming 828K and 386K tokens, respectively, exceeding the full pipeline.

The failures are consistent: repair patches inject prose into source files, core algorithms are omitted after repeated repair failures, and the configuration parser binds to an unreachable external service instead of using lightweight fallbacks. These results show that SCoT is a structured contract between high-level plans and executable

artifacts, providing the audit-repair loop with actionable signals that lightweight substitutes fail to preserve.

6.8.2 Impact of Shared Memory. We evaluate four shared-memory ablations, each removing a different component of cross-stage information flow. On SELFSTARTER, all variants exhibit the same failure pattern: apparent execution success with semantically incomplete outputs. Without Shared Memory, the paper context is reduced by two orders of magnitude, yielding a trivial script that succeeds by only checking for an output directory. Without Architecture Memory, the system produces a runnable scaffold but leaves core modules unimplemented. Without Structured Paper Memory, the parser relies on mocked behavior and generates hollow summaries despite consuming 1.1M tokens. Without Interface Memory, later modules drift from earlier interface commitments, causing failed configuration parsing to be treated as success. These results show that token efficiency alone is misleading in long-horizon agentic reproduction; the memory layer is essential for enforcing semantic completeness and cross-stage consistency under context constraints.

6.8.3 Impact of ARA. We ablate the two repair loops in the ARA: the static pass, which audits generated code for semantic consistency, and the dynamic pass, which executes the startup script in a sandbox and repairs runtime failures. On SELFSTARTER, the two passes address complementary errors. Without Static Repair, the code remains executable but suffers from semantic inaccuracies, such as imprecise ACL line matching and missed detections in no-clear-majority cases where devices report conflicting parameter values. Without Dynamic Repair, semantically plausible code fails at deployment due to sandbox-only issues, including Batfish port conflicts, missing container images, and orchestration failures. These results show that semantic correctness and executable deployment require distinct verification mechanisms, and that ARA’s two repair loops jointly convert borderline implementations into faithful and runnable reproductions.

7 discussion

7.1 Limitations

Fragility of TypedDict. TypedDict keeps data as plain dicts with no implicit behaviour binding, maintaining pipeline flexibility at the cost of having no runtime enforcement: schema violations — dropped Nones, renamed keys, mistyped fields — propagate until a runtime assert catches them, and the catch rate is bounded by the iteration cap, not by validation correctness.

Fragility of regex- and XML-based LLM response parsing. *RepLLM* uses a lightweight tag-based parser to extract JSON architectures, file blocks, and code patches from raw LLM responses. Even though this parser can handle common formatting deviations, including HTML-escaped tags and fenced-code fallbacks, its flexibility also makes the pipeline fragile. Stray closing tags, missing attributes, or explanatory prose may cause valid edits to be dropped or matched ambiguously. Constrained decoding or grammar-validated schemas could improve robustness.

Iteration caps and their justification. *RepLLM* bounds each repair loop with small fixed limits: the static-audit loop and the runtime-repair loop run at most three and five iterations, respectively. These limits are practical safeguards against non-progressing repair cycles. For example, malformed code patches may cause the patcher to repeatedly reject edits, preventing progress on the underlying bug. By failing fast and skipping stuck steps, *RepLLM* keeps the overall pipeline budget and allows downstream stages and final evaluation to proceed. The trade-off is that some recoverable bugs may be abandoned too early; we therefore prioritize system availability over exhaustive repair.

Scope of reproducible paper. In practice, *RepLLM* has limited support for three classes of systems papers: 1) *Kernel- and driver-level systems.* Papers that require patched kernels, custom eBPF programs, or GPU driver extensions are difficult to compile and test in the lightweight Docker sandbox provisioned by *ARA*. Their long build times, complex toolchains, and privileged execution requirements often prevent the audit loop from producing useful feedback. These failures are primarily environmental rather than algorithmic. 2) *Hardware-dependent and multi-node systems.* Papers that rely on specialized hardware or large-scale distributed deployments, such as custom NICs, accelerators, RDMA clusters, multi-rack BGP topologies, or geo-distributed key-value stores, cannot be fully exercised in the current single-node sandbox. Although *RepLLM* may generate structurally faithful code, performance claims involving throughput, tail latency, partition behavior, or hardware offloading remain unverifiable within the framework. 3) *Extensions to existing codebases.* Papers whose contributions are implemented as modifications to large external projects, such as Kubernetes scheduler hooks or LLVM passes, are not fully supported. *RepLLM* currently does not clone, pin, and patch the required upstream codebase. As a result, it may generate self-contained code that is structurally plausible but cannot compile against the intended project. This gap motivates future work on leveraging existing crawled codebases.

7.2 Future Work

Dynamic architecture refinement. *ARA* currently freezes the architecture once `arch.json` is generated. In future work, runtime evidence such as repeated `ModuleNotFoundError`s, persistent interface type mismatches, or consistently empty module outputs can be used to revise the architecture. We will add a lightweight re-entry point that re-prompts the architecture-design agent with accumulated failure logs, allowing it to propose module merges, splits, or interface revisions without restarting the full pipeline.

Leveraging existing crawled code. Many systems papers are more naturally reproduced by modifying an existing open-source repository than by generating code from scratch. We will augment the paper-parsing stage with optional repository URLs, clone and pin the target revision, and guide code generation to emit `<patch>` blocks against the cloned tree. Existing mechanisms for unique patch matching and safe path resolution can support this workflow, but they must be integrated with code search to identify the appropriate upstream files for modification.

Flexible sandbox construction. Current sandbox construction has limited support for code that depends on specific kernels, compilers, or hardware toolchains. To address this, we will augment the architecture agent with a custom `docker-compose.yml` or a Dockerfile with a non-default base image. The sandbox can then be provisioned from this explicit specification rather than from a heuristic runtime profile, enabling more environment-coupled artifacts to be built and tested while requiring additional validation of model-generated container specifications.

Shared failure memory. The current audit and runtime-repair steps are stateless across steps: once a repair attempt terminates, its failure trace is logged but not reused. We will introduce a per-paper failure memory keyed by paper, module, step, and error category. This memory can be consulted at the beginning of each repair pass to avoid repeated failed edits, such as `NOT_FOUND` or ambiguous `<old>` matches, and can provide prior successful fixes as in-context examples. This lightweight mechanism would allow the agent to learn from prior failures within the same reproduction task.

8 Conclusion

In this paper, we present *RepLLM*, a multi-agent framework for reproducing network research results from academic papers. *RepLLM* addresses the main barriers in this task: long and unstructured paper content, complex system logic, cross-file dependencies, and the lack of reliable verification. Instead of asking a single LLM to generate an entire codebase at once, *RepLLM* divides the workflow into content parsing, architecture design, code generation, and audit&repair. Its *Shared Memory* keeps paper information, interface definitions, intermediate states, and generated code in a consistent form across agents. Together with schema-first generation, structured reasoning, static analysis, and sandbox-based runtime feedback, *RepLLM* improves both executability and semantic alignment. Experiments on representative networking papers show that *RepLLM* produces more reliable reproductions than existing baselines while using fewer unnecessary tokens. More broadly, we hope *RepLLM* can motivate future work that further explores how LLM-based systems can support network research, not only in reproduction, but also in system design, validation, evaluation, and artifact generation.

Ethical Statement

This work does not raise any ethical issues.

Acknowledgments

We thank the anonymous reviewers for their valuable comments. This work is funded by the National Key R&D Program of China (Grant No. 2024YFB2906900), the National Natural Science Foundation of China (Grant Nos. 62532010, 62572408, and 62522203), and the Fujian Provincial Science and Technology Project (Grant No. 2025H6021).

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, USA, 265–283.
- [2] Firas Abuzaid, Srikanth Kandula, Behnaz Arzani, Ishai Menache, Matei Zaharia, and Peter Bailis. 2021. Contracting Wide-area Network Topologies to Solve Flow Problems Quickly. In *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI '21)*. USENIX Association, Berkeley, CA, USA, 175–200. <https://www.usenix.org/conference/nsdi21/presentation/abuzaid>
- [3] Anthropic. [n. d.]. Build with Claude: Context Windows. ([n. d.]). <https://platform.claude.com/docs/en/build-with-claude/context-windows> Accessed: 2026-07-03.
- [4] Anthropic. 2025. Claude Code - AI coding agent for terminal & IDE | Claude. <https://claude.com/product/claude-code>. (2025).
- [5] AnySphere. 2024. Cursor: An AI-Powered Code Editor. <https://cursor.com/>. (2024). Accessed: 2025-09.
- [6] Astral. 2026. Ruff: An extremely fast Python linter and code formatter. <https://github.com/astral-sh/ruff>. (2026).
- [7] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 31 (July 2024), 24 pages. <https://doi.org/10.1145/3643757>
- [8] Lukas Blecher, Guillem Cucurull Preixens, Thomas Scialom, and Robert Stojnic. 2024. Nougat: Neural Optical Understanding for Academic Documents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*, Vol. 2024. ICLR, 37646–37663. https://proceedings.iclr.cc/paper_files/paper/2024/file/a39a9aceda771cded859ae7560530e09-Paper-Conference.pdf
- [9] Huangxun Chen, Yukai Miao, Li Chen, Haifeng Sun, Hong Xu, Libin Liu, Gong Zhang, and Wei Wang. 2022. Software-defined network assimilation: bridging the last mile towards centralized network configuration management with NAssim. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 281–297. <https://doi.org/10.1145/3544216.3544244>
- [10] Xinyun Chen, Maxwell Lin, Nathanael Schaeli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*, Vol. 2024. ICLR, 8746–8825. https://proceedings.iclr.cc/paper_files/paper/2024/file/2460396f2d0d421885997dd1612ac56b-Paper-Conference.pdf
- [11] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. CROSSCODEVAL: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.), Vol. 36. Curran Associates Inc., Red Hook, NY, USA, 46701–46723. <https://doi.org/10.5555/3666122.3668145>
- [12] Docker Inc. 2013. Docker: Lightweight Linux Containers for Consistent Development and Operations. (2013). <https://www.docker.com/> Accessed: 2026-02-07.
- [13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [14] GitHub. 2022. GitHub Copilot. <https://github.com/features/copilot/>. (2022). Accessed: 2025-09.
- [15] Aryan Gulati. 2024. Controllable LLM Debugging: Knowing when to Stop Matters. (2024). https://cs191.stanford.edu/projects/Gulati,%20Aryan_NLP%20191W.pdf Stanford CS191/NLP 191W course project report. Accessed: 2026-07-03.
- [16] Nikhil Handigol, Brandon Heller, Vimalkumar Jayakumar, Bob Lantz, and Nick McKeown. 2012. Reproducible network experiments using container-based emulation. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '12)*. Association for Computing Machinery, New York, NY, USA, 253–264. <https://doi.org/10.1145/2413176.2413206>
- [17] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*, Vol. 2024. ICLR, 23247–23275. https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf
- [18] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*, Vol. 2024. ICLR, 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf
- [19] Xin Jin, Xiaozhou Li, Haoyu Zhang, Nate Foster, Jeongkeun Lee, Robert Soule, Changhoon Kim, and Ion Stoica. 2018. {NetChain}:{Scale-Free} {Sub-RTT} coordination. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*. USENIX Association, Berkeley, CA, USA, 35–49. <https://www.usenix.org/conference/nsdi18/presentation/jin>
- [20] Siva Kesava Reddy Kakarla, Ryan Beckett, Behnaz Arzani, Todd Millstein, and George Varghese. 2020. GRooT: Proactive Verification of DNS Configurations. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 310–328. <https://doi.org/10.1145/3387514.3405871>
- [21] Siva Kesava Reddy Kakarla, Alan Tang, Ryan Beckett, Karthick Jayaraman, Todd Millstein, Yuval Tamir, and George Varghese. 2020. Finding Network Misconfigurations by Automatic Template Inference. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*. USENIX Association, Berkeley, CA, USA, 999–1013. <https://www.usenix.org/conference/nsdi20/presentation/kakarla>
- [22] LangChain AI. 2022. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>. (2022). Accessed: 2026-02-06.
- [23] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Structured Chain-of-Thought Prompting for Code Generation. *arXiv preprint arXiv:2305.06599* (2023). <https://arxiv.org/abs/2305.06599>
- [24] Zongwei Li, Zhonghang Li, Zirui Guo, Xubin Ren, and Chao Huang. 2025. DeepCode: Open Agentic Coding. *arXiv preprint arXiv:2512.07921* (2025).
- [25] Zijie Lin, Yiqing Shen, Qilin Cai, He Sun, Jinrui Zhou, and Mingjun Xiao. 2025. AutoP2C: An LLM-Based Agent Framework for Code Repository Generation from Multimodal Content in Academic Papers. *arXiv preprint arXiv:2504.20115* (2025).
- [26] Jie Liu, Guohua Wang, Ronghui Yang, Jiajie Zeng, Mengchen Zhao, and Yi Cai. 2025. RTADev: Intention Aligned Multi-Agent Framework for Software Development. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 1548–1581. <https://doi.org/10.18653/v1/2025.findings-acl.80>
- [27] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638
- [28] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*, Vol. 2024. ICLR, 47832–47850. https://proceedings.iclr.cc/paper_files/paper/2024/file/893699c6824d4423b4092d4f898d00c7-Paper-Conference.pdf
- [29] Yi Luo, Linghang Shi, Yihao Li, Aobo Zhuang, Yeyun Gong, Ling Liu, and Chen Lin. 2025. From intention to implementation: automating biomedical research via LLMs. *Science China Information Sciences* 68, 7 (2025), 78–95. <https://doi.org/10.1007/s11432-024-4485-0>
- [30] Ansong Ni, Srinu Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *Proceedings of the 40th International Conference on Machine Learning (ICML '23)*, Vol. 202. PMLR, Cambridge, MA, USA, 26106–26128. <https://proceedings.mlr.press/v202/ni23.ni23.pdf>
- [31] OpenAI. 2022. ChatGPT. <https://chat.openai.com/>. (2022). Accessed: 2025-09.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32 (2019), 8026–8037. <https://doi.org/10.5555/3454287.3455008>
- [33] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. SynchroMesh: Reliable code generation from pre-trained language models. *arXiv preprint arXiv:2201.11227* (2022).
- [34] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL '24)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 15174–15186. <https://doi.org/10.18653/v1/2024.acl-long.810>
- [35] Kia Rahmani, Mohammad Raza, Sumit Gulwani, Vu Le, Daniel Morris, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2021. Multi-modal program inference: a marriage of pre-trained language models and component-based synthesis. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (Oct 2021), 1–29. <https://doi.org/10.1145/3485535>
- [36] Minju Seo, Jinheon Baek, Seongyun Lee, and Sung Ju Hwang. 2025. Paper2code: Automating code generation from scientific papers in machine learning. *arXiv*

- preprint arXiv:2504.17192* (2025).
- [37] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large Language Models Can Be Easily Distracted by Irrelevant Context. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.), Vol. 202. PMLR, 31210–31227. <https://proceedings.mlr.press/v202/shi23a.html>
 - [38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652. <https://doi.org/10.5555/3666122.3666499>
 - [39] Samuel Steffen, Timon Gehr, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2020. Probabilistic Verification of Network Configurations. In *Proceedings of the 2020 Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. ACM, New York, NY, USA, 750–764. <https://doi.org/10.1145/3387514.3405900>
 - [40] Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, Shengyan Liu, Di Luo, Yutao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, Shengzhu Yin, Minhui Zhu, Kilian Lieret, Yanxin Lu, Genglin Liu, Yufeng Du, Tianhua Tao, Ofir Press, Jamie Callan, Eliu Huerta, and Hao Peng. 2024. SciCode: A Research Coding Benchmark Curated by Scientists. *Advances in Neural Information Processing Systems* 37 (2024), 30624–30650. <https://doi.org/10.5555/3690531.3699047>
 - [41] Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, Zhiyuan Zhao, Rui Xu, Kaiwen Liu, Yuan Qu, Fukai Shang, Bo Zhang, Liqun Wei, Zhihao Sui, Wei Li, Botian Shi, Yu Qiao, Dahua Lin, and Conghui He. 2024. Mineru: An open-source solution for precise document content extraction. *arXiv preprint arXiv:2409.18839* (2024).
 - [42] Yibo Wang, Yunan Hou, Zeqi Lai, Hewu Li, Qian Wu, Jun Liu, Yuanjie Li, Xin Xie, and Zhifeng Han. 2025. How LLM Saved Me from Struggling with Experiment Reproduction: LEO Networking as A Case Study. In *Proceedings of the 2025 3rd Workshop on LEO Networking and Communication (LEO-NET '25)*. Association for Computing Machinery, New York, NY, USA, 1–7. <https://doi.org/10.1145/3748749.3749084>
 - [43] Zhijie Wang, Zijie Zhou, Da Song, Yuheng Huang, Shengmai Chen, Lei Ma, and Tianyi Zhang. 2025. Towards Understanding the Characteristics of Code Generation Errors Made by Large Language Models. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE '25)*. IEEE, Piscataway, NJ, USA, 2587–2599. <https://doi.org/10.1109/ICSE55347.2025.00180>
 - [44] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).
 - [45] Qiao Xiang, Yuling Lin, Mingjun Fang, Bang Huang, Siyong Huang, Ridi Wen, Franck Le, Linghe Kong, and Jiwei Shu. 2023. Toward Reproducing Network Research Results Using Large Language Models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets '23)*. Association for Computing Machinery, New York, NY, USA, 56–62. <https://doi.org/10.1145/3626111.3628189>
 - [46] Zhiying Xu, Francis Y Yan, Rachee Singh, Justin T Chiu, Alexander M Rush, and Minlan Yu. 2023. Teal: Learning-Accelerated Optimization of WAN Traffic Engineering. In *Proceedings of the ACM SIGCOMM 2023 Conference (SIGCOMM '23)*. ACM, New York, NY, USA, 378–393. <https://doi.org/10.1145/3603269.3604857>
 - [47] Lisa Yan and Nick McKeown. 2017. Learning Networking by Reproducing Research Results. *SIGCOMM Comput. Commun. Rev.* 47, 2 (May 2017), 19–26. <https://doi.org/10.1145/3089262.3089266>
 - [48] Hongkun Yang and Simon S. Lam. 2016. Real-Time Verification of Network Properties Using Atomic Predicates. *IEEE/ACM Transactions on Networking* 24, 2 (April 2016), 887–900. <https://doi.org/10.1109/TNET.2015.2398197>
 - [49] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. Intercode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. *Advances in Neural Information Processing Systems* 36 (2023), 23826–23854. <https://doi.org/10.5555/3666122.3668014>
 - [50] Lei Yang, Yossi Gilad, and Mohammad Alizadeh. 2024. Practical Rateless Set Reconciliation. In *Proceedings of the ACM SIGCOMM 2024 Conference (SIGCOMM '24)*. ACM, New York, NY, USA, 595–612. <https://doi.org/10.1145/3651890.3672219>
 - [51] Jane Yen, Tamás Lévai, Qinyuan Ye, Xiang Ren, Ramesh Govindan, and Barath Raghavan. 2021. Semi-Automated Protocol Disambiguation and Code Generation. In *Proceedings of the 2021 ACM SIGCOMM Conference (SIGCOMM '21)*. ACM, New York, NY, USA, 272–286. <https://doi.org/10.1145/3452296.3472910>
 - [52] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah Goodman, and Nick Haber. 2023. Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions. *Advances in Neural Information Processing Systems* 36 (2023), 31466–31523. <https://doi.org/10.5555/3666122.3668746>
 - [53] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP '23)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 2471–2484. <https://doi.org/10.18653/v1/2023.emnlp-main.152>
 - [54] Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 133 (April 2024), 25 pages. <https://doi.org/10.1145/3649850>
 - [55] Peng Zhang, Xu Liu, Hongkun Yang, Ning Kang, Zhengchang Gu, and Hao Li. 2020. {APKeep}: Realtime Verification for Real Networks. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*. USENIX Association, Berkeley, CA, USA, 241–255. <https://www.usenix.org/conference/nsdi20/presentation/zhang-peng>
 - [56] Xuanle Zhao, Zilin Sang, Yuxuan Li, Qi Shi, Weilin Zhao, Shuo Wang, Duzhen Zhang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2026. AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, USA, 21920–21942. <https://doi.org/10.18653/v1/2026.acl-long.1001>

Note: Appendices are supporting material that has not been peer-reviewed.

A Appendix

A.1 Detailed Analysis of Reproduction Performance Results

A.1.1 NetChain. We evaluate the end-to-end reproducibility of NetChain by testing whether different baselines, given identical inputs (paper, topology files, and command traces), can generate all four required artifacts (P4 data plane, Python control plane, client, and orchestration scripts) and successfully execute on a BMv2-based system.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction progressively covers the four-module NetChain architecture, including the P4 data plane, control-plane logic, client-side traffic generation, and orchestration scripts. The generated artifacts differ in the degree of client completeness, failure-handling logic, topology integration, and execution-environment alignment, but they collectively recover the main structural components of the system. The **human-corrected version** further enforces real Mininet/BMv2 execution, corrects lifecycle management and resource cleanup, and achieves successful end-to-end execution on the BMv2 testbed with correct packet exchange and consistent throughput across client and receiver logs.
- **Claude Code.** Across the low, medium, and high settings, Claude Code reconstructs NetChain as a host-side Python artifact rather than as a P4/BMv2 deployment. The lightweight version provides a quick single-script simulator, the medium version improves modularity with dataset loading and functional tests, and the high version shifts toward an analytical or closed-form performance model. These variants can reproduce parts of the experimental workflow or expected performance trends, but they do not generate the full deployable NetChain artifact set.
- **Standalone baseline.** Implements only a hand-written NetChain P4 data plane as a reference artifact, without control plane, client, or orchestration components, and therefore cannot execute end-to-end; it serves purely as a structural ground-truth baseline.

A.1.2 Rateless IBLT. We evaluate the reproducibility of Rateless IBLT by testing whether different reconstruction strategies, given identical inputs, can generate all required artifacts and successfully execute set-reconciliation experiments on a host-side Python runtime.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction consistently captures the core algorithmic structure of Rateless IBLT, including rateless mapping, encoding, peeling-style decoding, and set-difference reconciliation. The versions differ mainly in input-model fidelity, reconciliation procedure coverage, and whether tests exercise the real protocol path or simplified components. The **human-corrected version** replaces validation with tests

over the real Rateless IBLT protocol and achieves high consistency with the paper across mapping, encoding, peeling decoding, and set-difference reconciliation.

- **Claude Code.** Across the low, medium, and high settings, Claude Code produces Python Rateless IBLT simulators using the same synthetic dataset. The lightweight version runs end-to-end reconciliation at full scale and includes the Irregular Rateless IBLT variant; the medium version bundles the algorithm, dataset loading, benchmark execution, and report generation into a compact pipeline validated on a smaller sample; and the high version separates the core library from experiment drivers, emphasizing Monte Carlo reproduction of overhead curves and systematic medium-scale benchmarks. Overall, these variants reproduce the main algorithmic behavior but omit some production optimizations and application-level evaluations such as Ethereum state synchronization, multi-peer synchronization, adversarial workloads, SipHash, variable-length counts, and heap-based incremental encoding.
- **Standalone baseline.** The implementation reproduces the paper's core components and correctly supports data loading and encoding. Nevertheless, the current decoder does not maintain a record of recovered items, allowing the same item to be peeled and reinserted multiple times. This causes symbol states to oscillate between pure and non-pure, so that although very small instances terminate successfully, instances with $d \geq 10$ fail to complete. Consequently, the full evaluation cannot be executed, and the paper's central claims cannot currently be reproduced.

A.1.3 NCFlow. We evaluate the reproducibility of NCFlow by testing whether different reconstruction strategies, given identical WAN topology and traffic-demand inputs, can generate the required decomposition, routing, and optimization artifacts, and successfully execute multi-commodity flow experiments with third-party solver support.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction progressively covers NCFlow's decomposition, clustering, routing, and optimization pipeline. The generated artifacts vary in how strictly they align with the paper's baseline definitions, input formats, and offline/online execution structure, leading to different levels of agreement with the official implementation. The **human-corrected version** reaches near-identical total flow compared with the official implementation on the evaluated dataset, while remaining a streamlined single-dataset reproduction that does not cover larger topologies or the full experimental suite in the paper.
- **Claude Code.** Across the low, medium, and high settings, Claude Code implements solver-free simulations on the same dataset. The lightweight version focuses on quickly reproducing the core NCFlow logic on small topologies; the medium version improves modularity and adds metrics such as FIB

size while remaining stable on small and medium-scale inputs; and the high version provides a cleaner and more complete structure, although the experiment scale limits reproduction of the strongest speedups and extended scenarios reported in the paper.

- **Standalone baseline.** The standalone baseline lacks proper modular decomposition and orchestration and fails to provide an end-to-end evaluation pipeline.

A.1.4 GRoot. We evaluate the reproducibility of GRoot by testing whether different reconstruction strategies, given identical inputs (paper, metadata.json, jobs.json, and DNS zone files), can generate the full verification pipeline (zone parser, label graph, equivalence-class generator, symbolic executor, property checkers, and orchestration scripts) and correctly detect all reference violations on the 2 test benchmarks.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction covers the main components of the GRoot verification pipeline, including zone parsing, symbolic execution, equivalence-class construction, interpretation-graph modeling, and property checking. The versions differ in their treatment of equivalence-class refinement, name-server wiring, wildcard and DNAME semantics, apex handling, and checker alignment. The **human-corrected version** applies targeted fixes to these semantic and reporting components, aligns all seven test1 violation counts with output.json at 165 ECs and 19 JSON entries, and correctly detects the five-hop rewrite chain in test2.
- **Claude Code.** Across the low, medium, and high settings, Claude Code produces host-side Python DNS verifiers with increasing structural completeness. The low version quickly implements zone parsing, equivalence-class generation, and property checking; the medium version modularizes the pipeline and adds layered DNS semantics with broader checker coverage; and the high version introduces formal DNS models, symbolic execution, and more complete property checking. These variants recover important parts of the benchmark behavior, but still show varying degrees of mismatch in equivalence-class counts, violation counts, and entry-level reporting.
- **Standalone baseline.** Provides a hand-written single-file Python DNS verifier with basic zone parsing, simplified equivalence-class generation, and trace-based property checks, but omits full interpretation-graph construction and benchmark-aligned reporting, under-detects most test datasets reference violations and leaves several expected failure categories uncovered.

A.1.5 NetDice. We evaluate the reproducibility of NetDice by testing whether different reconstruction strategies, given identical inputs (paper, JSON configurations, and topology files), can generate the full analysis pipeline and match reference results on the test dataset.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction covers parsing, routing simulation, property checking, failure exploration, and probabilistic analysis to different degrees. The versions vary in the integration of pipeline stages, hot-edge reasoning, ECMP handling, and

bounded exploration. The **human-corrected version** applies targeted fixes to these components and passes all 11 tests on the same dataset, with probabilities on simple examples, ECMP, and NSFnet benchmarks matching reference values within tolerance.

- **Claude Code.** Across the low, medium, and high settings, Claude Code produces host-side Python probabilistic analyzers over test datasets. The lightweight version implements a single-script pipeline for parsing, BGP simulation, hot-edge identification, and failure exploration; the medium version modularizes dataset loading and batch execution, using exhaustive enumeration on small networks and bounded exploration on larger topologies; and the high version implements NetDice's core algorithms and property types with cold-edge pruning, batch evaluation, and brute-force cross-checks. The high setting achieves the closest agreement with reference probabilities, especially on ECMP and larger-topology benchmarks.
- **Standalone baseline.** A single-file NetDice analyzer covering static routing, independent link failures, and basic property checking on embedded mock input, without BGP simulation, dataset loading, or the full multi-stage pipeline, and therefore cannot support standard end-to-end evaluation; it serves purely as a minimal algorithmic ground-truth baseline.

A.1.6 SelfStarter. We evaluate the reproducibility of SELFSTARTER by testing whether different implementations can generate the required template-inference artifacts and run structured-generalization experiments from the same router-configuration inputs. SELFSTARTER detects likely misconfigurations by inferring templates for ACLs, prefix lists, and route policies, and identifying structural or parameter outliers across nodes.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction focuses on the ACL template-inference workflow, covering configuration ingestion, block-based structured generalization, line matching, metatemplate construction, group partitioning, parameter mapping, and outlier reporting. The versions progressively improve the organization of the ACL pipeline, including staged execution and Batfish-oriented ingestion, while route-map and prefix-list templating remain outside the main evaluated path. The **human-corrected version** reproduces the expected evaluation profile on the primary test dataset: seven ACL segments are templated, four are structurally consistent, two group outliers from missing-line structural differences are correctly identified, one parameter-level discrepancy is flagged for manual review, and the final summary reports two group-consistency failures and one parameter-consistency failure, with all other consistency checks passing.
- **Claude Code.** Across the low, medium, and high settings, Claude Code produces Python SELFSTARTER structured-generalization simulators over the same synthetic benchmark. The lightweight version runs end-to-end ACL-template inference on seven ACL configurations from two routers; the medium version integrates Cisco IOS parsing, dataset loading, and benchmark execution into a self-contained pipeline

with field-level parameterization; and the high version modularizes parsing, matching, inference, and output generation while adding systematic benchmarking and finer-grained octet-level LGG parameterization. All three focus primarily on ACL structured generalization.

- **Standalone baseline.** The standalone implementation independently reproduces SELFSTARTER as a self-contained system and supports the core structured-generalization workflow for Cisco IOS ACLs. It successfully parses all seven ACLs from two routers in the test1 dataset and reproduces the expected templates, parameter mappings, and outlier classifications. Nevertheless, the implementation relies on a lightweight regular-expression parser and only supports extended ACLs, while routing-policy and prefix-list templating are not implemented. Consequently, test2 cannot be completed because incorrect execution parameters cause segment matching to return an empty result.

A.1.7 Teal. We evaluate the end-to-end reproducibility of Teal by testing whether different implementations can train and evaluate path-formulation traffic allocations from the same WAN topologies, traffic matrices, and candidate-path inputs. Teal accelerates wide-area traffic engineering by combining a flow-centric graph neural network to encode topology and demand features, multi-agent reinforcement learning to allocate each traffic demand toward a global objective, and ADMM fine-tuning to reduce link-capacity violations across nodes.

- **ReplLM.** Across the automatically generated ReplLM versions, the reconstruction establishes Teal’s main train-and-evaluate framework, including topology and traffic-matrix ingestion, candidate-path handling, FlowGNN-style encoding, COMA-style policy training, ADMM-based post-processing, metric aggregation, logging, checkpoints, and allocation export. The versions differ in the completeness of model integration, graph/tensor conversion, inference consistency, ADMM formulation, and large-topology scalability. The **human-corrected version** reproduces the expected evaluation profile on the primary datasets: on **test/B4**, it loads real traffic matrices and cached candidate paths, trains the FlowGNN-COMA-ADMM stack end-to-end, outputs non-negative split ratios with per-demand sums not exceeding one, and reports total flow, satisfied-demand ratio, maximum link utilization, capacity violations, and sub-second per-interval runtime, with measurable ADMM improvement over the policy-only allocation; on **test/UsCarrier**, it completes training and evaluation with the larger-topology ADMM iteration budget when path caches are available. Kdl and ASN2k remain outside the default audit path because absent path caches make on-the-fly generation infeasible on CPU, and full-scale training on these topologies requires substantially more GPU memory than the lightweight smoke setting provides.
- **Claude Code.** Across the low, medium, and high settings, Claude Code produces Python Teal traffic-allocation simulators implementing FlowGNN-style encoding, multi-agent reinforcement learning, and ADMM fine-tuning over the same WAN traffic-engineering datasets. The lightweight version completes end-to-end training and evaluation on B4

with full COMA policy-gradient training and ADMM; the medium version bundles data loading, simplified COMA training, ADMM, and report generation into a single self-contained module; and the high version separates the core library from experiment drivers, emphasizing per-interval benchmarks, ADMM ablations, structured metric export, and UsCarrier forward-pass checks. These variants reproduce the B4 workflow most completely, while large-scale end-to-end training, LP/NCFlow baselines, link-failure experiments, and GPU speedups remain outside the reproduced scope.

- **Standalone baseline.** Standalone attempts to reproduce Teal’s high-level framework, including path-based traffic engineering, candidate-path generation, FlowGNN, COMA-based training, and simplified ADMM fine-tuning. However, the implementation cannot complete dataset loading or execution due to data-format assumptions, hard-coded dimensional errors, and indexing bugs, so even the limited B4 evaluation cannot be reproduced, and the paper’s scalability and performance claims remain unsupported.

A.2 Prompts of LLM-based Evaluation

We provide the prompts for LLM-based global-level semantic alignment evaluation in Figure 6.

Prompt for evaluating the generated code

You are a software reproduction auditor. Your task is to assess how much of the system or method described in the paper is actually realized in the reproduced code, and how faithfully those realized parts follow the paper description.

You will be provided with:

- the complete reproduced code,
- the associated data structure definitions,
- and the original method description from the paper.

Your evaluation objective is to determine the degree of coverage and fidelity of the reproduction.

Scoring should primarily reflect how many paper-described components are implemented and how accurately they are implemented, rather than how many deviations can be identified.

Global Evaluation Principles (Must Be Followed)

- Apply a coverage-first principle: implemented and verifiable components contribute positively to the score.
- Missing components should not be treated as explicit errors, but as absence of positive evidence.
- Explicitly implemented components that deviate from the paper description should reduce the score.
- The maximum achievable score is proportional to the breadth of the paper method that is actually implemented.
- Superficial similarity (e.g., names or comments) does not count as implementation.
- Scores of 9–10 require both broad coverage and high fidelity, and should remain rare.

Evaluation Dimensions**#### 1. Correctness**

For the components that are implemented, how faithfully do they follow the paper's described logic, execution order, and constraints?

Evaluation should focus only on components that can be clearly identified in the code.

Scoring logic:

- Start from a baseline determined by the number of correctly implemented core components.
- Increase the score as more components are implemented with high fidelity.
- Reduce the score only when an implemented component clearly contradicts or violates the paper description.

Scoring guidance:

- 0–2: Very few identifiable components are implemented
- 3–4: Some components are implemented, but with major deviations
- 5–6: Multiple core components are implemented with partial fidelity
- 7–8: Most implemented components closely follow the paper
- 9–10: Nearly all implemented components are faithful to the paper description

2. Completeness

To what extent does the code cover the components, stages, and execution paths described in the paper?

Completeness should be evaluated as coverage, not penalty.

Scoring logic:

- Identify the set of core components described in the paper.
- Assess how many of these components are meaningfully implemented in the code.
- Absence of a component limits the maximum score but does not directly reduce it.

Scoring guidance:

- 0–2: Only a minimal subset of the paper method is implemented
- 3–4: A small but non-trivial portion of components is implemented
- 5–6: Core structure is present, but coverage is partial
- 7–8: Most core components are implemented
- 9–10: All or nearly all core components are implemented

Figure 6: Prompts for LLM-based Paper–Code Semantic Alignment Evaluation

Prompt for evaluating the generated code

3. Readability and Maintainability

How clearly does the code expose the implemented components and their correspondence to the paper description?
Readability should be evaluated in terms of traceability rather than style alone.

Scoring logic:

- Higher scores require that implemented components can be easily mapped to the paper.
- Poor structure or naming reduces the ability to gain credit for implemented components.

Scoring guidance:

- 0–2: Implemented components are difficult to identify or verify
- 3–4: Components are identifiable but require significant effort to trace
- 5–6: Most implemented components can be reasonably traced
- 7–8: Clear structure enables straightforward paper-to-code mapping
- 9–10: Code structure directly reflects the paper’s organization

Output Requirements

Return exactly one JSON code block and do not output any additional text.

```
```json
{
 "evaluation": {
 "Correctness": {
 "score": X,
 "justification": "Describe which paper components are implemented and how faithfully they follow the paper."
 },
 "Completeness": {
 "score": X,
 "justification": "Describe which core components are implemented and which are not, focusing on coverage."
 },
 "Readability_Maintainability": {
 "score": X,
 "justification": "Explain how easily the implemented components can be traced to the paper."
 }
 },
 "overall_score": X.X,
 "summary": "A coverage-oriented summary emphasizing what has been implemented well and what remains unimplemented."
}
```
```

Input Data

The code is: [[code/]]

The data structure is: [[data_structure/]]

The related paper content is: [[paper_content]]

The entry point file is [[entry_file]]